

测试计划 (GB8567—88)

1 引言

1.1 编写目的

本测试计划用于指导ROS2智能药品拣选系统的测试工作。主要目的是：

- 对系统各个功能模块进行全面测试
- 验证系统是否满足功能需求和质量指标
- 确保AI医疗助手、药房后端、权限认证等各组件的集成质量
- 识别和记录系统缺陷，为质量改进指导方向

本计划适用于系统开发团队、质量保证部门及项目管理人员。

1.2 背景

系统名称：ROS2智能药品拣选系统

项目历史与现状：

- 本项目是一个集成智能医疗助手与自动化药房后端的系统
- 系统包含多个模块：AI医疗助手（P1/Agent模块）、自动化药房后端API、权限认证系统、智能筛选系统、ROS2集成
- 核心功能包括药品库存管理、智能用药推荐、医生审批 workflow、机器人自动化取药
- 已完成的工作：基础架构搭建、核心API开发、P1医疗助手系统、权限认证系统、部分集成测试

测试环境要求：

- Python 3.8+ 环境
- 一个配置的LLM API密钥
- SQLite数据库
- ROS2环境（可选，用于ROS集成测试）
- Flask服务器运行环境

1.3 定义

- **LLM**：大语言模型（Large Language Model）
- **API**：应用程序接口（Application Programming Interface）
- **RBAC**：基于角色的访问控制（Role-Based Access Control）

- **ROS2**: 机器人操作系统第二版 (Robot Operating System 2)
- **REST**: 表现层状态转化 (Representational State Transfer)
- **端到端测试**: E2E测试, 从用户操作到系统响应的完整流程测试
- **集成测试**: 验证不同模块间的协作正确性
- **单元测试**: 对单个函数或类的独立测试
- **CRUD**: 创建 (Create)、读取 (Read)、更新 (Update)、删除 (Delete) 操作

1.4 参考资料

- Python官方文档 - <https://docs.python.org/3/>
- pytest测试框架文档 - <https://docs.pytest.org/>
- Flask文档 - <https://flask.palletsprojects.com/>

2 计划

2.1 软件说明

本系统是一个集成式的智能药品拣选系统，主要功能模块如下：

模块名称	功能描述	主要输入	主要输出	质量指标
AI医疗助手	症状分析、药品推荐、剂量计算、多药联用检查	患者症状、基本信息	用药建议、相关注意事项	准确率 >95%，响应时间 <5s
药品管理API	CRUD药品、库存管理、过期药品清理	药品数据、操作指令	操作结果、药品列表	可用性 >99%，准确率100%
订单管理	创建、查询、更新、删除订单	订单信息	订单状态、订单详情	准确率 100%，响应时间 <1s
审批工作流	医生审批用药建议、审计日志	待审批项、审批意见	审批结果、审计记录	准确率 100%，可审计性完整
权限认证系统	JWT认证、RBAC权限控制	用户凭证、权限请求	令牌、权限验证结果	安全性高，认证成功率高 100%

模块名称	功能描述	主要输入	主要输出	质量指标
智能筛选系统	药品智能匹配、症状同义词扩展	症状关键词、匹配规则	匹配药品列表	召回率>90%，准确率>85%
ROS2集成	任务发布、机器人状态监控	取药任务、系统状态	执行反馈、机器人状态	集成成功率>95%
库存管理	实时库存跟踪、有效期管理	库存变化、过期药品	库存更新、清理结果	数据一致性100%

2.2 测试内容

本测试计划包含以下测试内容，分为四个主要阶段：

测试阶段	测试名称	测试类型	优先级	依赖条件
阶段1：单元测试	UT-001-药品API基础操作	单元测试	高	无
	UT-002-LLM集成测试	单元测试	高	需要API Key
	UT-003-权限认证系统	单元测试	高	无
	UT-004-智能筛选模块	单元测试	中	需要药品数据库
	UT-005-工具函数测试	单元测试	中	无
阶段2：API集成测试	IT-001-药品API集成	集成测试	高	后端运行
	IT-002-订单API集成	集成测试	高	后端运行
	IT-003-审批流程集成	集成测试	高	后端运行

测试阶段	测试名称	测试类型	优先级	依赖条件
	IT-004-权限认证集成	集成测试	高	后端运行
	IT-005-智能筛选集成	集成测试	中	后端运行
阶段3：端到端测试	E2E-001-完整用药流程	端到端测试	高	完整环境
	E2E-002-医生审批流程	端到端测试	高	完整环境
	E2E-003-多角色权限验证	端到端测试	中	完整环境
	E2E-004-ROS2机器人集成	端到端测试	中	ROS2环境
阶段4：性能与压力测试	PT-001-并发请求测试	性能测试	中	后端运行
	PT-002-数据库性能测试	性能测试	中	后端运行
	PT-003-响应时间测试	性能测试	中	后端运行

2.3 测试1：单元测试 - 药品API基础操作 (UT-001)

给出这项测试内容的参与单位及被测试的部位。

参与单位：质量保证部门、开发部门（Python开发者）

被测试部位： `agent_with_backend/api/drug_controller.py` 、数据库连接模块

2.3.1 进度安排

日期	工作内容	工时	负责人
Day 1	环境准备、依赖安装	1h	测试工程师
Day 1-2	编写单元测试用例	4h	测试工程师
Day 2	执行测试、收集结果	2h	测试工程师
Day 2-3	缺陷分析、报告	2h	测试Lead

2.3.2 条件

a. 设备需求:

- 物理设备: 测试PC (CPU>2核, 内存>4GB)
- 并发测试PC: 可选

b. 软件需求:

- Python 3.8+ 环境
- pytest 测试框架
- pytest-cov 覆盖率插件
- SQLite3 数据库
- Flask Web框架
- 支持库: requests、pandas等 (见requirements.txt)

c. 人员需求:

- 1名测试工程师: 编写和执行测试用例、答疑和问题解决
- 1名测试Lead: 审核和报告

2.3.3 测试资料

- [测试用例列表](#): 已有的基础药品API测试代码
- [项目API文档](#): API接口规范
- [数据库初始化脚本](#): 测试数据准备
- [示例请求](#): API调用示例
- [测试驱动程序](#): pytest固定装置

2.3.4 测试培训

- **内容**: pytest框架基础、单元测试最佳实践、项目代码结构、API接口说明
- **受训人员**: 全体QA工程师、参与测试的开发人员
- **培训方式**: 文档学习 (2h) + 实机演练 (1h)
- **原资料**:
 - [pytest官方文档](#)
 - [项目开发文档](#)
 - [P1医疗助手系统README](#)

2.4 测试2: API集成测试 - 药品API集成 (IT-001)

用与本测试计划2.3条相类似的方式说明用于另一项常规API集成测试的测试工作计划。

参与单位: 质量保证部门、后端开发部门

被测试部位: `agent_with_backend/api/drug_controller.py` (完整的HTTP API)、数据库持久化、业务逻辑

2.4.1 进度安排

日期	工作内容	工时	负责人
Day 3	后端启动、数据初始化	1h	DevOps/测试工程师
Day 3-4	编写API集成测试用例	5h	高级QA工程师
Day 4	执行集成测试（首轮）	3h	QA工程师
Day 4-5	缺陷排查、二次测试	2h	QA工程师
Day 5	生成报告	1h	测试Lead

2.4.2 条件

a. 设备需求：

- 后端服务器：Flask运行环境
- 测试客户端：用于发送HTTP请求

b. 软件需求：

- Python 3.8+ 环境
- Flask、Flask-CORS、Flask-SQLAlchemy
- requests HTTP客户端库
- pytest + pytest-flask（用于Flask应用测试）
- 数据库：SQLite（带相关驱动）
- ROS2集成库（可选）

c. 人员需求：

- 1名高级QA工程师（5人天）：设计集成测试用例、执行
- 1名后端开发人员（2人天）：环境配置、问题排查
- 1名DevOps工程师（1人天）：服务启动、配置管理

2.4.3 测试资料

- [API测试辅助函数](#)：通用HTTP请求库
- [现有API集成测试](#)：参考测试实现
- [药品API手册](#)：API完整规范
- [订单API集成测试](#)：相关测试参考
- [完整集成测试示例](#)：端到端流程参考
- [后端配置文档](#)：配置说明

2.4.4 测试培训

- **内容：** REST API设计、测试HTTP API的方法、集成测试最佳实践、项目API规范讲解、常见问题处理
- **受训人员：** API集成测试的全体QA人员、相关后端开发人员
- **培训方式：** 在线讲座（1h） + 动手实验室（2h）)
- **原资料：**
 - [Postman API测试指南](#)
 - [项目API集成文档](#)
 - [现有测试用例分析](#)

2.5 测试3：端到端测试 - 完整用药流程（E2E-001）

参与单位： 质量保证部门、全栈开发团队
被测试部位： AI医疗助手（P1）、后端API、数据库、权限系统、ROS2集成，以及这些组件间的完整交互

2.5.1 进度安排

日期	工作内容	工时	负责人
Day 6	完整环境部署（AI+后端+ROS2）	2h	DevOps/系统架构师
Day 6-7	准备测试用例和测试数据	3h	高级QA工程师
Day 7-8	端到端流程执行（多次迭代）	8h	QA团队
Day 8-9	缺陷验证和问题排查	4h	QA工程师+开发人员
Day 9	性能指标记录、报告生成	2h	测试Lead

2.5.2 条件

- a. 设备需求：**
- 完整集成测试环境PC/服务器：CPU >= 4核，内存 >= 8GB
 - ROS2运行环境（可选）
 - 模拟患者和医生的多个客户端
- b. 软件需求：**
- 完整的系统栈：AI医疗助手（P1） + Flask后端 + SQLite数据库 + ROS2（可选）

- Python 3.8+，所有依赖包
- 一个配置的LLM API密钥（OpenAI或Claude）
- 测试执行工具：pytest、curl、postman或类似工具
- 可选：日志收集工具、监控工具

c. 人员需求：

- 1名高级QA工程师（5人天）：设计端到端场景、主导测试执行
- 2名QA工程师（6人天）：参与测试执行
- 1-2名后端开发人员（3人天）：环境维护、问题排查
- 1名系统架构师（1人天）：完整流程设计咨询
- 1名测试Lead（2人天）：监督、报告

2.5.3 测试资料

- [完整集成测试脚本](#)：参考实现
- [P1医疗助手CLI测试](#)：交互式测试工具
- [系统架构文档](#)：完整流程说明
- [后端启动说明](#)：环境启动
- [ROS2集成说明](#)：ROS2测试指南

2.5.4 测试培训

- **内容**：系统完整架构、端到端测试方法论、多模块协同测试、问题诊断技巧、性能监控和指标收集
- **参训人员**：全体参与质量保证的人员（QA、开发、架构师等）
- **培训方式**：架构讲座（2h）+ 实地演练（4h）+ 问题排查工作坊（2h）
- **原资料**：
 - [系统架构分析文档](#)
 - [workflow说明](#)
 - [快速开始指南](#)

2.6 测试4：ROS2集成测试 - 机器人集成（E2E-004）

参与单位：质量保证部门、ROS2集成开发人员、机器人工程师

被测试部位：ROS2消息发布订阅、任务消息格式转换、机器人状态监控、错误处理与降级机制

2.6.1 进度安排

日期	工作内容	工时	负责人
Day 10	ROS2环境搭建、节点测试	3h	ROS2工程师

日期	工作内容	工时	负责人
Day 10-11	编写ROS2集成测试	4h	QA工程师+ROS开发者
Day 11	消息格式测试、话题监控	3h	QA工程师
Day 11-12	故障模拟、降级机制验证	3h	QA工程师+开发
Day 12	性能测试、报告	2h	测试Lead

2.6.2 条件

a. 设备需求：

- ROS2运行环境（Ubuntu 20.04或22.04）
- 模拟机器人环境或真实机器人
- ROS2话题监控工具

b. 软件需求：

- ROS2（完整发行版）
- ROS2 Python客户端库
- rclpy消息转换工具
- pytest ROS2扩展
- 系统Python环境中的ROS2集成代码

c. 人员需求：

- 1名ROS2工程师（3人天）：环境搭建、ROS2测试支持
- 1名QA工程师（4人天）：集成测试用例编写和执行
- 1名后端开发人员（2人天）：问题排查
- 1名机器人工程师（可选）（1人天）：机器人行为验证

2.6.3 测试资料

- [ROS2集成文档](#)：详细测试步骤
- [ROS2适配代码](#)：集成实现
- [测试ROS节点示例](#)：测试工具
- [配置文档](#)：ROS2配置

2.6.4 测试培训

- **内容**：ROS2基础概念、消息发布-订阅机制、话题监控、错误处理、性能调优
- **受训人员**：QA工程师、后端开发人员、ROS2工程师
- **培训方式**：ROS2官方文档学习（2h）+ 项目代码讲解（1.5h）+ 上机实验（2h）

- **原资料：**
 - [ROS2官方文档](#)
 - [项目ROS2集成指南](#)
-

3 测试设计说明

3.1 测试1：单元测试 - 药品API基础操作 (UT-001)

说明对第一项测试内容的测试设计考虑。

3.1.1 控制

控制方式：完全自动化测试

- **输入方式：**自动生成测试数据，通过Python代码直接调用被测函数
- **控制顺序：**按测试类别组织，单一职责原则，一个函数测一个功能
- **结果记录方式：**pytest自动记录、生成HTML报告，结合覆盖率工具

具体步骤：

1. 使用pytest discovery自动发现测试文件
2. 对每个测试函数执行前置固定装置 (setup) 初始化测试环境
3. 执行测试逻辑，记录任何异常
4. 对每个测试函数执行后置固定装置 (teardown) 清理环境
5. 生成覆盖率报告

3.1.2 输入

输入数据说明：

- **药品数据：**
 - 标准药品：名称、分类、库存量、有效期、货架位置
 - 边界数据：最小库存、最大库存、过期日期为今天的药品
 - 异常数据：重复ID、特殊字符名称、无效库存（负数）
- **操作指令：**
 - 有效操作：创建(CREATE)、读取(READ)、更新(UPDATE)、删除(DELETE)
 - 边界操作：批量操作、并发操作、操作不存在的药品
 - 异常操作：未授权操作、超出限制的操作

数据选择策略：

- 使用等价类划分：有效输入类、边界输入类、无效输入类
- 采用边界值分析：对于数值类字段取边界值
- 随机生成一部分测试数据以发现未预见的问题

3.1.3 输出

预期输出：

- **成功操作：**
 - 返回HTTP 200-201状态码
 - 返回结构化JSON响应，包含 `{success: true, data: {...}, message: "..."}`
 - 数据库状态同步变化（可通过查询验证）
- **错误情况：**
 - 对应的HTTP错误码（400/404/500等）
 - 错误响应结构： `{success: false, error_type: "...", message: "..."}`
 - 数据库保持一致性（失败时回滚）
- **日志输出：**
 - 请求日志：操作时间、操作人、操作类型
 - 错误日志：错误堆栈、上下文信息

3.1.4 过程

测试执行步骤：

1. 初始化阶段 (setup_method)

- 创建测试数据库副本
- 初始化Flask应用测试客户端
- 种子数据加载

2. 创建(CREATE)测试

- 输入：新药品对象（名称、分类、库存等）
- 执行：调用API POST /drugs
- 验证：返回201, 获取ID, 数据库中能查到
- 异常处理：验证重复ID时返回400

3. 读取(READ)测试

- 输入：已知的药品ID、查询参数（按症状、分类等）
- 执行：调用API GET /drugs/{id}或GET /drugs?symptom=...
- 验证：返回200，数据完整准确
- 异常处理：验证不存在ID返回404

4. 更新(UPDATE)测试

- 输入：药品ID、更新内容（库存、有效期等）
- 执行：调用API PUT /drugs/{id}

- 验证：返回200，数据库中的数据已更新，其他字段不变

5. 删除(DELETE)测试

- 输入：药品ID、删除原因
- 执行：调用API DELETE /drugs/{id}
- 验证：返回204，数据库中数据已删除，查询返回404

6. 清理阶段 (teardown_method)

- 清理测试数据库
- 关闭数据库连接
- 恢复系统状态

运行命令：

```
pytest tests/test_drug_api.py -v --cov=api.drug_controller --cov-report=html
```

3.2 测试2：API集成测试 - 药品API集成 (IT-001)

用与本测试计划3.1条相类似的方式说明第2项及其后各项测试工作的设计考虑。

3.2.1 控制

控制方式：自动化集成测试，针对完整的HTTP API

- **输入方式：**通过HTTP请求 (requests库或curl) 调用API端点
- **控制顺序：**模拟实际的业务流程顺序，例如先创建则读取再更新再删除
- **结果记录方式：**pytest记录HTTP状态码、响应体、响应时间，记入报告

具体步骤：

1. 启动Flask应用 (test server)
2. 对每个API集成测试用例，准备先决条件 (如必要的初始数据)
3. 发送HTTP请求，记录响应时间和响应内容
4. 验证响应状态码、响应头、响应体结构
5. 验证副作用 (数据库变化、审计日志等)
6. 清理测试数据

3.2.2 输入

输入方式：

- **HTTP请求行：**
 - 方法：GET、POST、PUT、DELETE、PATCH
 - URL： `http://localhost:8001/api/drugs` 、 `/api/orders` 等

- Headers: `Content-Type: application/json`、授权令牌

- 请求体示例:

```
{
  "drug_name": "阿司匹林",
  "category": "解热镇痛药",
  "quantity": 100,
  "unit": "片",
  "expiry_date": "2025-12-31"
}
```

- 查询参数:

- `?symptom=头痛&limit=10&offset=0` (分页)
- `?drug_id=123` (特定查询)

数据选择策略:

- 标准流程: 创建→查询→更新→删除
- 错误场景: 403权限不足、400参数错误、500服务器错误
- 并发场景: 多个客户端同时修改同一资源

3.2.3 输出

预期输出:

- HTTP响应:

- 状态码范围: 2xx(成功)、4xx(客户端错误)、5xx(服务器错误)
- 响应头: `Content-Type: application/json`
- 响应体: 所有字段按合同规范返回

- 响应时间:

- 查询单条: <100ms
- 列表查询: <500ms
- 创建/更新: <200ms

- 数据库一致性:

- 创建后能立即查询
- 更新后再次查询值已改变
- 删除后查询返回404

3.2.4 过程

测试执行步骤:

1. 环境准备

- 启动Flask应用: `python agent_with_backend/main.py`
- 等待服务就绪 (health check)
- 清理旧测试数据

2. 药品创建集成测试

- 步骤: `POST /api/drugs` 新建阿司匹林
- 验证: 200状态码, 返回`drug_id`
- 副作用验证: 数据库`drug`表新增1条记录

3. 药品查询集成测试

- 步骤: `GET /api/drugs?symptom=头痛`
- 验证: 200状态码, 结果列表包含阿司匹林
- 分页验证: 检查`total`、`has_next`字段

4. 库存更新集成测试

- 步骤: `PUT /api/drugs/{id}` 设置库存为50
- 验证: 200状态码, 再次GET确认库存为50
- 审计日志验证: `audit_log`表有新记录

5. 权限校验集成测试

- 步骤: 不提供授权令牌, 尝试删除药品
- 验证: 403 Forbidden响应

6. 并发修改集成测试

- 步骤: 两个请求同时更新同一药品的库存
- 验证: 最终状态一致, 无数据丢失

运行命令:

```
export PHARMACY_BASE_URL=http://localhost:8001
pytest tests/test_drug_api.py -v -s # -s 显示print输出
```

3.3 测试3: 端到端测试 - 完整用药流程 (E2E-001)

3.3.1 控制

控制方式: 真实业务场景模拟

- 参与者模拟:
 - 患者: 通过CLI或API提交症状
 - 医生: 通过审批API批准用药建议
 - 系统: 自动处理数据库更新、ROS2发布等
- 控制流程:

1. 患者描述症状 → 2. Agent分析药品 → 3. 医生审批
4. 库存更新 → 5. ROS2发布任务 → 6. 验证完整性

3.3.2 输入

业务场景输入：

- 患者信息：

```
{  
  "patient_name": "王先生",  
  "age": 30,  
  "weight": 60,  
  "symptoms": "头痛、发烧",  
  "allergies": "无",  
  "current_medications": []  
}
```

- 医生审批操作：批准/驳回、审批意见

3.3.3 输出

完整业务流程的预期结果：

- 患者获得用药建议
- 医生收到审批请求
- 审批通过后库存自动扣减
- ROS2话题收到取药任务消息
- 完整的审计日志记录全过程

3.3.4 过程

完整用药流程测试步骤：

```
# 步骤1: 启动完整系统
启动Flask后端 + P1 Agent

# 步骤2: 患者提交症状
POST /api/patients/consultation
{
  "symptoms": "头痛",
  "age": 30,
  "weight": 60,
  "patient_name": "Wang"
}

# 步骤3: 验证Agent处理
- 症状解析成结构化数据
- 检索相关药品
- 生成用药建议

# 步骤4: 医生审批
POST /api/approvals/{recommendation_id}/approve
{
  "doctor_id": 1,
  "approved": true,
  "comments": "同意"
}

# 步骤5: 验证库存更新
GET /api/drugs/aspirin
assert response['quantity'] < original_quantity

# 步骤6: 验证ROS2消息（如有）
监听ROS2话题 /task_request 或 /task_topic
验证收到取药任务

# 步骤7: 审计日志检查
GET /api/audit_logs?recommendation_id={id}
验证包含所有操作记录
```

3.4 测试4: ROS2集成测试 - 机器人集成 (E2E-004)

3.4.1 控制

控制方式: ROS2话题监控和消息流验证

- **监听话题:** 使用 `ros2 topic echo` 或测试节点监听系统发布的消息

- **消息验证**：验证消息格式、内容正确性

3.4.2 输入

ROS2消息输入/场景：

- 系统发起取药请求 → 发送task消息到ROS2
- 机器人状态变化 → 接收state消息

3.4.3 输出

ROS2消息验证输出：

- 验证 `/task_request` 或 `/task_topic` 话题收到消息
- 验证消息格式符合协议
- 验证错误处理与降级机制

3.4.4 过程

ROS2集成测试步骤：

```
# 步骤1：启动ROS2环境
source /opt/ros/humble/setup.bash
ros2 daemon start

# 步骤2：启动话题监听
ros2 topic echo /task_request &
ros2 topic info /task_topic

# 步骤3：触发系统中的取药流程
# 通过审批API或直接调用获得ROS2发布

# 步骤4：验证消息到达
# 检查echo输出中是否包含预期的取药任务信息

# 步骤5：验证降级机制
# 模拟ROS2不可用，验证系统日志记录而不崩溃
export ROS2_INTEGRATION_MODE=LOG_ONLY # 降级到仅记录日志模式
# 重复步骤3-4，验证系统继续工作
```

4 评价准则

4.1 范围

测试覆盖范围： 本测试计划覆盖以下范围内的功能模块：

模块	覆盖范围	覆盖率目标	备注
药品管理API	CRUD操作、查询筛选、库存管理	> 90%	优先级高
订单管理API	订单创建、查询、更新、状态管理	> 85%	优先级高
审批 workflow	审批提交、审批处理、审计日志	> 85%	优先级高
权限认证系统	用户登录、JWT令牌、角色权限	> 90%	优先级高
智能筛选系统	药品匹配、同义词扩展、搜索功能	> 80%	优先级中
AI 医疗助手 (P1)	症状分析、药品推荐、剂量计算	> 80%	外部LLM依赖
ROS2集成	消息发布、降级处理	> 85%	需要ROS2环境
数据库操作	CRUD、事务、约束检查	> 90%	优先级高
错误处理	异常捕获、错误响应、日志记录	> 85%	优先级中
并发处理	并发访问、锁定、数据一致性	> 75%	可选深度测试

覆盖率局限性：

- ROS2集成测试：需专用ROS2环境，可能在多种配置下进行
- LLM集成：依赖第三方API，无法完全模拟所有情况
- 机器人硬件集成：使用模拟器替代，不涉及真实硬件行为

4.2 数据整理

数据处理方法：

1. 自动化数据整理

- pytest覆盖率插件自动生成 `htmlcov/index.html` 报告
- 测试结果自动汇总为JUnit XML格式
- 日志自动收集到日志文件

2. 手工数据处理

- 性能指标（响应时间）：从测试日志中提取，计算平均值、P95、P99
- 缺陷统计：分类统计的缺陷优先级、状态分布
- 覆盖率数据：从HTML报告中提取关键指标

3. 数据转换技术

运行单元测试并生成覆盖率报告：

```
pytest tests/ --cov=agent_with_backend --cov-report=html --cov-report=te
```

生成JUnit格式报告（CI/CD集成）：

```
pytest tests/ --junit-xml=test-results.xml
```

性能数据提取（使用pytest-benchmark）：

```
pytest tests/ --benchmark-only --benchmark-json=benchmark.json
python scripts/parse_benchmark.py benchmark.json
```

缺陷跟踪数据导出（从测试失败信息）：

```
# 在测试报告中自动识别和分类缺陷
# 按优先级、模块、错误类型分类
```

4.3 尺度

测试通过准则：

4.3.1 单元测试通过准则

准则项	判断条件	量化指标
功能正确性	所有测试用例均成功执行	成功率 = 100%
代码覆盖率	语句覆盖、分支覆盖、路径覆盖	语句覆盖 ≥ 90%, 分支覆盖 ≥ 85%
时间约束	单元测试执行时间	<10秒
缺陷数量	严重缺陷（P1）数	P1缺陷数 = 0
	主要缺陷（P2）数	P2缺陷数 ≤ 2
	次要缺陷（P3）数	P3缺陷数 ≤ 5

判定： P1缺陷必须修复后重测， P2缺陷必须修复或风险评估后， P3缺陷可接受。

4.3.2 集成测试通过准则

准则项	判断条件	量化指标
功能完整性	API调用-返回完整链路可用	≥95% API端点可用
接口一致性	响应格式符合规范	100% 符合合同
数据一致性	数据库与API返回数据一致	一致性>99%
响应时间	API响应满足SLA	查询<500ms, 更新<200ms
缺陷等级	集成测试中发现的缺陷等级	P1=0, P2≤1, P3≤3
测试覆盖	API覆盖度	≥85% API接口

4.3.3 端到端测试通过准则

准则项	判断条件	量化指标
业务流程完整	完整流程可以执行完成	完成率 ≥ 95%
用户体验	界面（若有）可用性	所有关键操作可正常执行
性能指标	端到端响应时间	完整流程 <10秒
健壮性	在不同数据/场景下的稳定性	成功率 ≥ 95%
缺陷等级	E2E测试中的缺陷	P1=0, P2≤2, P3≤5

4.3.4 ROS2集成测试通过准则

准则项	判断条件	量化指标
话题可用性	ROS2话题正确发布	发布成功率 ≥ 98%
消息格式	消息内容符合定义	格式正确率 100%
降级机制	系统降级处理正常	降级成功率 100%
错误恢复	ROS2故障后恢复	恢复成功率 ≥ 95%

4.3.5 系统整体通过准则

所有测试全部完成，且满足以下条件时，系统可进行验收：

1. 缺陷准入

- P1缺陷：必须为0
- P2缺陷：≤3个，且具体分析和风险评估
- P3缺陷：≤10个，可延迟修复

2. 性能准入

- 单个API响应时间 <1秒
- 完整业务流程 <10秒
- 数据库查询 <500ms

3. 质量准入

- 代码覆盖率 ≥85%
- 集成测试通过率 ≥95%
- 端到端测试通过率 ≥95%
- ROS2集成成功率 ≥95% (如适用)

4. 安全准入

- 权限认证测试通过
- 数据加密传输验证通过
- 审计日志记录完整

4.4 缺陷优先级定义

缺陷分类标准：

优先级	定义	示例	修复期限
P1 (严重)	功能完全不可用或数据丢失	药品查询API完全返回错误、删除药品后无法恢复、审批流程无法启动	必须立即修复
P2 (主要)	功能部分不可用或性能严重下降	某个特定场景下的查询错误、响应时间>2秒、特定用户权限无效	必须在发布前修复
P3 (次要)	功能可用但存在缺陷或性能轻微下降	UI显示不一致、错误提示不清楚、响应时间1-2秒	可延迟修复

5 测试工具与环境

5.1 测试工具清单

工具类别	工具名称	版本	用途
测试框架	pytest	7.0+	单元测试、集成测试执行
	pytest-cov	3.0+	代码覆盖率统计
	pytest-flask	1.2+	Flask应用测试
	pytest-asyncio	0.18+	异步测试支持
HTTP测试	requests	2.27+	API调用
	curl	7.0+	命令行API测试
	Postman	最新版	API可视化测试
性能测试	pytest-benchmark	3.4+	性能基准测试
	locust	2.0+	压力负载测试
ROS2工具	ros2 CLI	humble+	ROS2话题监控
	rclpy	3.0+	ROS2 Python客户端
数据库	sqlite3	3.35+	测试数据库
代码质量	pylint	2.12+	代码静态分析
	black	22.0+	代码格式化

5.2 测试环境配置

5.2.1 硬件环境

- 最低配置：CPU 2核，内存 4GB，存储 20GB
- 推荐配置：CPU 4核，内存 8GB，存储 50GB
- 并发测试：CPU 8核，内存 16GB（可选）

5.2.2 软件环境

操作系统: Ubuntu 20.04 LTS 或 Ubuntu 22.04 LTS

Python: 3.8 或 3.9 或 3.10 或 3.11

数据库: SQLite 3.35+

依赖管理: pip 或 conda

虚拟环境: venv 或 conda env

ROS2: Humble (可选)

5.2.3 环境搭建步骤

1. 克隆项目

```
git clone <repo_url>
```

```
cd ROS2-based-Intelligent-Medicine-Pickup-System/agent_with_backend
```

2. 创建虚拟环境

```
python -m venv venv
```

```
source venv/bin/activate
```

3. 安装依赖

```
pip install -r requirements.txt
```

```
pip install pytest pytest-cov
```

4. 初始化数据库

```
python database/scripts/init_db.py
```

```
python database/scripts/seed_drugs.py
```

5. 设置环境变量

```
cp .env.example .env
```

编辑.env, 填入LLM API Key等配置

6. 验证环境

```
pytest --version
```

```
python -c "import flask; print(f'Flask {flask.__version__}')"

```

```
curl http://localhost:8001/api/health
```